

MLShield: Complete Technical Blueprint

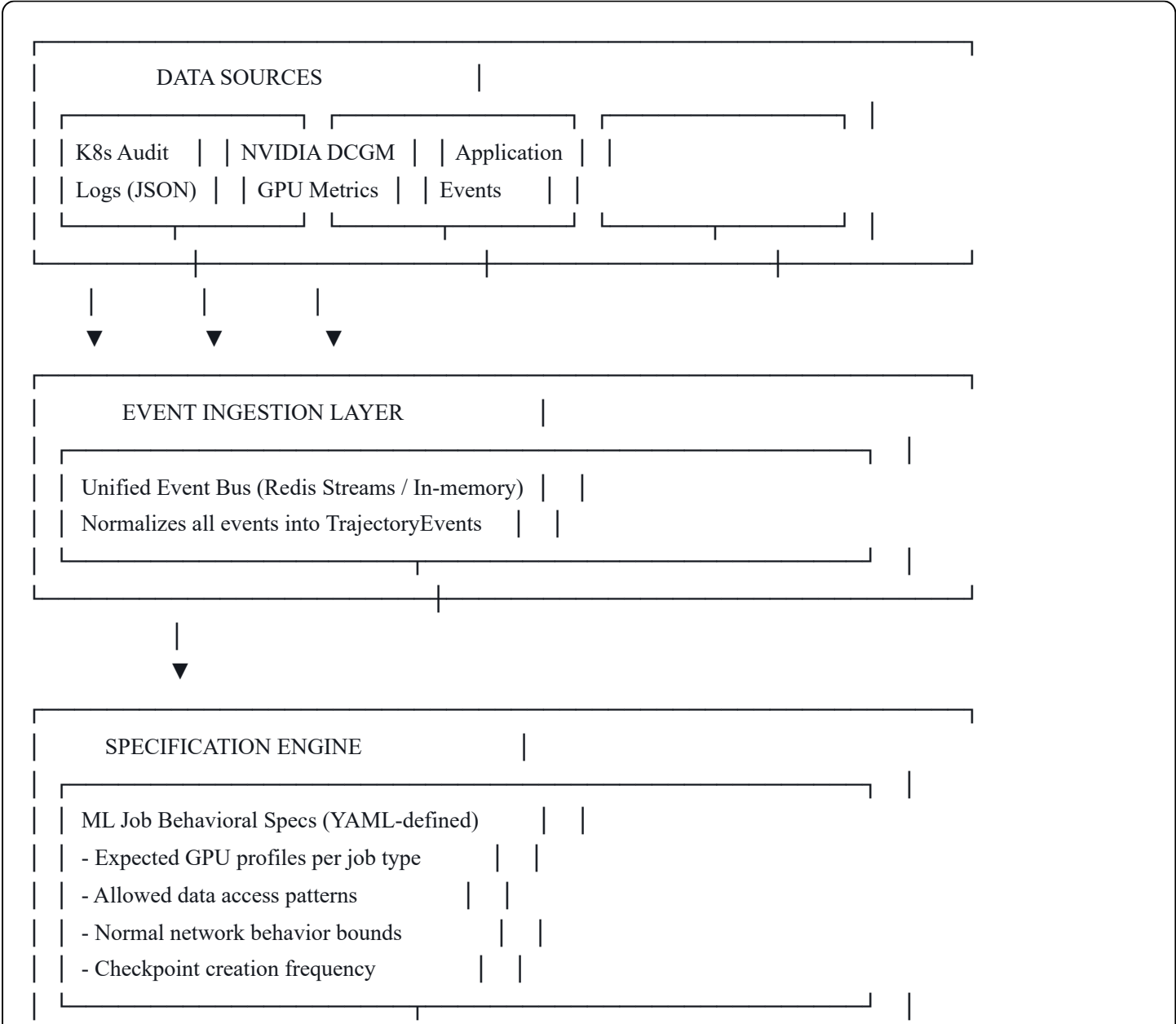
ML-Infrastructure-Aware Anomaly Detection for Weight Exfiltration

1. WHAT YOU'RE BUILDING

MLShield is an open-source Python tool that monitors Kubernetes-based ML training clusters for model weight exfiltration. It adapts StepShield's cascaded HybridGuard architecture from code-agent trajectories to infrastructure event sequences.

Core capability: Ingest K8s audit logs + NVIDIA GPU telemetry → define behavioral specifications for ML training jobs → detect specification violations using a 3-layer cascade → report temporal security metrics (Early Intervention Rate, Detection Gap, Damage Prevented).

2. ARCHITECTURE OVERVIEW



- └─ LICENSE # Apache 2.0
- └─ pyproject.toml # Project config + dependencies
- └─ Dockerfile # Container for MLShield itself
- └─ docker-compose.yml # Full demo stack
- └─ Makefile # Common commands
- |
- └─ configs/
 - └─ default_specs.yaml # Default ML job behavioral specs
 - └─ rules/
 - └─ weight_access.yaml # Rules for weight file access
 - └─ gpu_anomaly.yaml # Rules for GPU usage anomalies
 - └─ network_exfil.yaml # Rules for network exfiltration
 - └─ data_staging.yaml # Rules for suspicious data staging
 - └─ demo_config.yaml # Demo environment config
- |
- └─ src/
 - └─ mlshield/
 - └─ __init__.py
 - └─ main.py # Entry point
 - |
 - └─ ingestion/ # Data source connectors
 - └─ __init__.py
 - └─ base.py # Abstract ingestion interface
 - └─ k8s_audit.py # Kubernetes audit log ingester
 - └─ dcmg_metrics.py # NVIDIA DCGM GPU metrics ingester
 - └─ app_events.py # Application-level event ingester
 - └─ event_bus.py # Unified event normalization
 - |
 - └─ specs/ # Behavioral specification engine
 - └─ __init__.py
 - └─ spec_parser.py # YAML spec parser
 - └─ spec_validator.py # Runtime spec validation
 - └─ job_profiles.py # ML job behavioral profiles
 - └─ spec_types.py # Spec data classes
 - |
 - └─ detectors/ # 3-layer cascaded detector
 - └─ __init__.py
 - └─ cascade.py # Orchestrates the 3 layers
 - └─ layer1_rules.py # Static rule engine
 - └─ layer2_ml.py # LSTM + Isolation Forest
 - └─ layer3_llm.py # LLM semantic judge
 - └─ models/
 - └─ lstm_detector.py # LSTM sequence model
 - └─ isolation.py # Isolation Forest wrapper
 - |
 - └─ metrics/ # Temporal security metrics
 - └─ __init__.py

- └─ temporal.py # EIR, Detection Gap, Damage Prevented
- └─ prometheus.py # Prometheus metrics exporter
- └─ evaluator.py # Benchmark evaluation
-
- └─ api/ # FastAPI dashboard + REST API
 - └─ __init__.py
 - └─ app.py # FastAPI application
 - └─ routes/
 - └─ alerts.py # Alert management endpoints
 - └─ metrics.py # Metrics endpoints
 - └─ specs.py # Spec management endpoints
 - └─ health.py # Health check
 - └─ templates/
 - └─ dashboard.html # Simple monitoring dashboard
- └─ utils/
 - └─ __init__.py
 - └─ logging.py # Structured logging
 - └─ config.py # Configuration management
- └─ benchmark/ # Synthetic benchmark dataset
 - └─ generate_dataset.py # Dataset generator
 - └─ scenarios/
 - └─ normal_training.py # Normal training job trajectories
 - └─ weight_exfiltration.py # Weight theft attack trajectories
 - └─ data_poisoning.py # Training data poisoning
 - └─ model_extraction.py # API-based model extraction
 - └─ unauthorized_distill.py # Unauthorized distillation
 - └─ shadowray_style.py # ShadowRay-style cluster attack
- └─ data/ # Generated datasets go here
 - └─ evaluate.py # Run evaluations against benchmark
- └─ deploy/ # Deployment manifests
 - └─ kubernetes/
 - └─ namespace.yaml
 - └─ mlshield-deployment.yaml
 - └─ mlshield-service.yaml
 - └─ falco-rules-cm.yaml # Custom Falco rules ConfigMap
 - └─ dcgm-exporter.yaml
 - └─ helm/
 - └─ mlshield/ # Helm chart (stretch goal)
- └─ tests/
 - └─ test_ingestion.py
 - └─ test_specs.py
 - └─ test_detectors.py
 - └─ test_metrics.py

```
| └─ test_cascade.py
|
└─ notebooks/
    │ └─ 01_explore_k8s_audit_logs.ipynb
    │ └─ 02_gpu_telemetry_analysis.ipynb
    │ └─ 03_train_lstm_detector.ipynb
    └─ 04_benchmark_evaluation.ipynb
|
└─ docs/
    │ └─ ARCHITECTURE.md
    │ └─ THREAT_MODEL.md           # Formal threat taxonomy
    │ └─ SPECS_FORMAT.md          # How to write behavioral specs
    └─ CONTRIBUTING.md
```

4. TECH STACK & DEPENDENCIES

Core Dependencies (pyproject.toml)

```
toml
```

```
[project]
name = "mlshield"
version = "0.1.0"
description = "ML-Infrastructure-Aware Anomaly Detection for Weight Exfiltration"
requires-python = ">=3.10"
```

```
dependencies = [
    # Core
    "fastapi>=0.104.0",
    "uvicorn>=0.24.0",
    "pydantic>=2.5.0",
    "pyyaml>=6.0",
    "redis>=5.0.0",

    # ML Detection (Layer 2)
    "torch>=2.1.0",
    "scikit-learn>=1.3.0",
    "numpy>=1.24.0",
    "pandas>=2.1.0",

    # Kubernetes
    "kubernetes>=28.1.0",

    # Monitoring
    "prometheus-client>=0.19.0",

    # HTTP client (for LLM API calls)
    "httpx>=0.25.0",

    # Logging & Utils
    "structlog>=23.2.0",
    "rich>=13.7.0",
    "click>=8.1.0",
]
```

```
[project.optional-dependencies]
```

```
dev = [
    "pytest>=7.4.0",
    "pytest-asyncio>=0.21.0",
    "black>=23.0",
    "ruff>=0.1.0",
    "mypy>=1.7.0",
]
```

External Infrastructure (docker-compose.yml for demo)

```
yaml

version: '3.8'

services:
  # MLShield core service
  mlshield:
    build: .
    ports:
      - "8000:8000" # FastAPI dashboard
      - "9090:9090" # Prometheus metrics
    volumes:
      - ./configs:/app/configs
    environment:
      - MLSHIELD_CONFIG=/app/configs/demo_config.yaml
      - LLM_API_URL=https://api.anthropic.com/v1/messages
    depends_on:
      - redis

  # Event bus
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  # Simulated K8s audit log source (for demo)
  audit-simulator:
    build: ./benchmark
    environment:
      - REDIS_URL=redis://redis:6379
      - SCENARIO=mixed

  # Prometheus (optional, for metrics visualization)
  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9091:9090"
    volumes:
      - ./deploy/prometheus.yml:/etc/prometheus/prometheus.yml

  # Grafana (optional, for dashboards)
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
```

5. IMPLEMENTATION DETAILS

5.1 Event Data Model

python


```

# src/mlshield/ingestion/event_bus.py
from dataclasses import dataclass, field
from enum import Enum
from datetime import datetime
from typing import Optional

class EventSource(Enum):
    K8S_AUDIT = "k8s_audit"
    DCGM_GPU = "dcgm_gpu"
    APP_EVENT = "app_event"
    FALCO_ALERT = "falco_alert"

class EventSeverity(Enum):
    INFO = "info"
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    CRITICAL = "critical"

@dataclass
class TrajectoryEvent:
    """Unified event format — all data sources normalize to this."""
    event_id: str
    timestamp: datetime
    source: EventSource
    job_id: str          # K8s job/pod identifier
    user: Optional[str]  # User or service account
    action: str          # What happened (e.g., "read_checkpoint", "gpu_util_spike")
    resource: str        # What was acted on (e.g., "model-v3.pt", "gpu-0")
    details: dict = field(default_factory=dict) # Source-specific metadata
    trajectory_step: int = 0 # Step number within this job's trajectory

@dataclass
class Trajectory:
    """Ordered sequence of events for a single ML job."""
    job_id: str
    events: list[TrajectoryEvent] = field(default_factory=list)
    start_time: Optional[datetime] = None
    job_type: str = "unknown" # training, inference, data_pipeline
    spec_name: Optional[str] = None # Which behavioral spec applies

    def add_event(self, event: TrajectoryEvent):
        event.trajectory_step = len(self.events)
        self.events.append(event)

```

```
if not self.start_time:  
    self.start_time = event.timestamp
```

5.2 Kubernetes Audit Log Ingestor

```
python
```

```

# src/mlshield/ingestion/k8s_audit.py
import json
import asyncio
from pathlib import Path
from typing import AsyncGenerator
from datetime import datetime
from .event_bus import TrajectoryEvent, EventSource

# ML-security-relevant K8s audit events
ML_RELEVANT_RESOURCES = {
    "pods", "jobs", "secrets", "configmaps",
    "persistentvolumeclaims", "services", "deployments",
    "serviceaccounts", "roles", "rolebindings"
}

# Actions that matter for weight security
SECURITY_RELEVANT_VERBS = {
    "get", "list", "watch", "create", "update",
    "patch", "delete", "exec", "attach", "portforward"
}

class K8sAuditIngester:
    """Ingests Kubernetes audit logs and normalizes to TrajectoryEvents."""

    def __init__(self, log_source: str = "/var/log/kubernetes/audit.log"):
        self.log_source = log_source
        self.checkpoint_patterns = [
            "checkpoint", "model", ".pt", ".pth", ".bin",
            ".safetensors", ".onnx", "weights"
        ]

    async def stream_events(self) -> AsyncGenerator[TrajectoryEvent, None]:
        """Stream audit events from the K8s audit log."""
        # In production: tail the audit log file or connect to webhook
        # For demo: read from file or Redis stream
        async for line in self._tail_log():
            event = self._parse_audit_event(line)
            if event and self._is_ml_relevant(event):
                yield self._to_trajectory_event(event)

    def _parse_audit_event(self, raw: str) -> dict | None:
        try:
            return json.loads(raw)
        except json.JSONDecodeError:
            return None

```

```

def _is_ml_relevant(self, event: dict) -> bool:
    """Filter to ML-security-relevant events."""
    resource = event.get("objectRef", {}).get("resource", "")
    verb = event.get("verb", "")
    return (
        resource in ML_RELEVANT_RESOURCES
        and verb in SECURITY_RELEVANT_VERBS
    )

def _to_trajectory_event(self, audit: dict) -> TrajectoryEvent:
    """Convert K8s audit event to unified TrajectoryEvent."""
    obj_ref = audit.get("objectRef", {})
    user_info = audit.get("user", {})

    # Detect weight-related access
    resource_name = obj_ref.get("name", "")
    is_weight_access = any(
        p in resource_name.lower() for p in self._checkpoint_patterns
    )

    return TrajectoryEvent(
        event_id=audit.get("auditID", ""),
        timestamp=datetime.fromisoformat(
            audit.get("requestReceivedTimestamp", "")
            .replace("Z", "+00:00")
        ),
        source=EventSource.K8S_AUDIT,
        job_id=self._extract_job_id(audit),
        user=user_info.get("username"),
        action=f"k8s_{audit.get('verb', 'unknown')}",
        resource=f"{obj_ref.get('resource', '')}/{resource_name}",
        details={
            "namespace": obj_ref.get("namespace", ""),
            "api_group": obj_ref.get("apiGroup", ""),
            "response_code": audit.get("responseStatus", {}).get("code"),
            "source_ip": (audit.get("sourceIPs") or ["unknown"])[0],
            "is_weight_access": is_weight_access,
            "user_agent": audit.get("userAgent", ""),
        }
    )

def _extract_job_id(self, audit: dict) -> str:
    """Extract a meaningful job identifier from the audit event."""
    obj_ref = audit.get("objectRef", {})
    ns = obj_ref.get("namespace", "default")
    name = obj_ref.get("name", "unknown")
    return f"{ns}/{name}"

```

```
async def _tail_log(self) -> AsyncGenerator[str, None]:
    """Tail the audit log file (like tail -f)."""
    path = Path(self.log_source)
    if not path.exists():
        # Demo mode: generate synthetic events
        from benchmark.scenarios.normal_training import generate_normal_events
        for event in generate_normal_events():
            yield json.dumps(event)
            await asyncio.sleep(0.1)
        return

    with open(path) as f:
        f.seek(0, 2) # Go to end
        while True:
            line = f.readline()
            if line:
                yield line.strip()
            else:
                await asyncio.sleep(0.5)
```

5.3 NVIDIA DCGM GPU Metrics Ingester

python

```

# src/mlshield/ingestion/dcgm_metrics.py
import httpx
import asyncio
from datetime import datetime
from .event_bus import TrajectoryEvent, EventSource

# Key GPU metrics for security monitoring
SECURITY_RELEVANT_METRICS = [
    "DCGM_FI_DEV_GPU_UTIL",    # GPU utilization %
    "DCGM_FI_DEV_MEM_COPY_UTIL", # Memory utilization %
    "DCGM_FI_DEV_FB_USED",    # Framebuffer memory used (MiB)
    "DCGM_FI_DEV_FB_FREE",    # Framebuffer memory free (MiB)
    "DCGM_FI_DEV_NVLINK_BANDWIDTH_TOTAL", # NVLink bandwidth
    "DCGM_FI_DEV_PCIE_REPLAY_COUNTER", # PCIe errors
    "DCGM_FI_DEV_GPU_TEMP",    # Temperature
    "DCGM_FI_DEV_POWER_USAGE", # Power draw
    "DCGM_FI_DEV_SM_CLOCK",    # SM clock frequency
    "DCGM_FI_DEV_ENC_UTIL",    # Encoder utilization (data movement!)
]

class DCGMIngester:
    """Ingest NVIDIA GPU telemetry from DCGM Exporter's Prometheus endpoint."""

    def __init__(
        self,
        dcgm_url: str = "http://localhost:9400/metrics",
        poll_interval: float = 5.0,
    ):
        self.dcgm_url = dcgm_url
        self.poll_interval = poll_interval
        self._baselines: dict[str, dict] = {} # gpu_id -> baseline metrics
        self._history: dict[str, list] = {} # gpu_id -> metric history

    async def stream_events(self):
        """Poll DCGM metrics and emit events on anomalies."""
        async with httpx.AsyncClient() as client:
            while True:
                try:
                    response = await client.get(self.dcgm_url)
                    metrics = self._parse_prometheus(response.text)

                    for gpu_id, gpu_metrics in metrics.items():
                        # Update history
                        if gpu_id not in self._history:
                            self._history[gpu_id] = []
                        self._history[gpu_id].append(gpu_metrics)

```

```

# Build baseline from first N samples
if len(self._history[gpu_id]) == 20:
    self._baselines[gpu_id] = self._compute_baseline(
        self._history[gpu_id]
    )

# Check for anomalies against baseline
if gpu_id in self._baselines:
    anomalies = self._check_anomalies(
        gpu_id, gpu_metrics
    )
    for anomaly in anomalies:
        yield anomaly

# Always emit a periodic snapshot event
yield TrajectoryEvent(
    event_id=f"dcgm-{gpu_id}-{datetime.utcnow().timestamp()}",
    timestamp=datetime.utcnow(),
    source=EventSource.DCGM_GPU,
    job_id=gpu_metrics.get("pod_name", f"gpu-{gpu_id}"),
    user=None,
    action="gpu_metrics_snapshot",
    resource=f"gpu/{gpu_id}",
    details=gpu_metrics,
)

except httpx.RequestError:
    pass # DCGM not available, skip this cycle

await asyncio.sleep(self.poll_interval)

```

```

def _parse_prometheus(self, text: str) -> dict:
    """Parse Prometheus text format into structured metrics."""
    metrics = {}
    for line in text.strip().split("\n"):
        if line.startswith("#") or not line:
            continue
        # Parse: METRIC_NAME{labels} value
        try:
            metric_part, value = line.rsplit(" ", 1)
            if "{" in metric_part:
                name, labels_str = metric_part.split("{", 1)
                labels_str = labels_str.rstrip("}")
                labels = dict(
                    item.split("=")
                    for item in labels_str.replace("'", "").split(",")
                )
            else:
                labels = {}
            metrics[metric_part] = value
        except ValueError:
            continue

```

```

        if "=" in item
    )
else:
    name = metric_part
    labels = {}

    gpu_id = labels.get("gpu", "0")
    if gpu_id not in metrics:
        metrics[gpu_id] = {"pod_name": labels.get("pod", "")}
    metrics[gpu_id][name] = float(value)
except (ValueError, KeyError):
    continue
return metrics

def _compute_baseline(self, history: list[dict]) -> dict:
    """Compute mean and std for each metric from history."""
    import numpy as np
    baseline = {}
    for key in SECURITY_RELEVANT_METRICS:
        values = [h.get(key, 0) for h in history if key in h]
        if values:
            baseline[key] = {
                "mean": np.mean(values),
                "std": np.std(values),
                "min": np.min(values),
                "max": np.max(values),
            }
    return baseline

def _check_anomalies(self, gpu_id: str, current: dict) -> list:
    """Check if current metrics deviate significantly from baseline."""
    events = []
    baseline = self._baselines[gpu_id]

    for metric_name, stats in baseline.items():
        value = current.get(metric_name)
        if value is None:
            continue

        # Flag if > 3 standard deviations from baseline
        if stats["std"] > 0:
            z_score = abs(value - stats["mean"]) / stats["std"]
            if z_score > 3.0:
                events.append(TrajectoryEvent(
                    event_id=f"anomaly-{gpu_id}-{metric_name}-{datetime.utcnow().timestamp()}",
                    timestamp=datetime.utcnow(),
                    source=EventSource.DCGM_GPU,

```



```
job_id=current.get("pod_name", f"gpu-{gpu_id}"),
user=None,
action=f"gpu_anomaly_{metric_name.lower()}",
resource=f"gpu/{gpu_id}",
details={
    "metric": metric_name,
    "value": value,
    "baseline_mean": stats["mean"],
    "baseline_std": stats["std"],
    "z_score": z_score,
    "severity": "high" if z_score > 5 else "medium",
},
))
return events
```

5.4 Behavioral Specification Engine

yaml

```
# configs/default_specs.yaml
```

```
# MLShield Behavioral Specifications for ML Training Jobs
```

```
specs:
```

```
- name: standard_training
  description: "Standard model training job behavioral profile"
  job_type: training
  match:
    labels:
      app: training
    namespace_pattern: "ml-*
```

```
allowed_behaviors:
```

```
# Who can access what
```

```
data_access:
```

```
  allowed_paths:
```

- "/data/training/*"
- "/models/checkpoints/*"

```
  denied_paths:
```

- "/models/production/*" # No access to prod weights
- "/secrets/*"

```
  allowed_operations: ["read", "write"]
```

```
# Expected GPU profile
```

```
gpu_profile:
```

```
  utilization_range: [60, 100] # Expected GPU util %
  memory_range_pct: [40, 95] # Expected VRAM usage %
  expected_gpu_count: [1, 8]
  max_nvlink_bandwidth_spike: 3.0 # x baseline
```

```
# Network behavior
```

```
network:
```

```
  allowed_egress:
```

- ".*.internal.cluster"
- "registry.internal"

```
  denied_egress:
```

- ".*.s3.amazonaws.com" # No direct S3 uploads
- ".*.blob.core.windows.net" # No Azure blob uploads
- ".*huggingface.co" # No HF uploads during training

```
  max_egress_rate_mbps: 100
```

```
# Checkpoint behavior
```

```
checkpoints:
```

```
  max_frequency_minutes: 10 # No more than 1 checkpoint per 10 min
  allowed_formats: [".pt", ".safetensors"]
  max_size_gb: 50
```

must_be_encrypted: **true**

What triggers an alert

violations:

- **type:** weight_access_outside_pipeline

description: "Model weights accessed outside approved training pipeline"

severity: critical

condition: "data_access to /models/ from non-training pod"

- **type:** unusual_gpu_pattern

description: "GPU utilization doesn't match declared job type"

severity: high

condition: "gpu_util < 20% for training job (possible distillation setup)"

- **type:** suspicious_egress

description: "Outbound network to unauthorized destination"

severity: critical

condition: "egress to denied destination"

- **type:** checkpoint_anomaly

description: "Abnormal checkpoint creation pattern"

severity: medium

condition: "checkpoint frequency > max or format not in allowed list"

python

```

# src/mlshield/specs/spec_validator.py
import yaml
import re
from dataclasses import dataclass
from typing import Optional
from ..ingestion.event_bus import TrajectoryEvent

@dataclass
class ViolationResult:
    """Result of checking an event against a spec."""
    is_violation: bool
    violation_type: str = ""
    severity: str = "info"
    description: str = ""
    spec_name: str = ""
    event: Optional[TrajectoryEvent] = None
    step_number: int = 0

class SpecValidator:
    """Validates events against behavioral specifications."""

    def __init__(self, spec_path: str = "configs/default_specs.yaml"):
        with open(spec_path) as f:
            self.config = yaml.safe_load(f)
            self.specs = {s["name"]: s for s in self.config.get("specs", [])}

    def validate_event(
        self,
        event: TrajectoryEvent,
        spec_name: str = "standard_training"
    ) -> ViolationResult:
        """Check a single event against its behavioral spec."""
        spec = self.specs.get(spec_name)
        if not spec:
            return ViolationResult(is_violation=False)

        # Check each violation type
        checks = [
            self._check_data_access(event, spec),
            self._check_network_egress(event, spec),
            self._check_gpu_anomaly(event, spec),
            self._check_checkpoint_behavior(event, spec),
        ]

        # Return the most severe violation found
        violations = [c for c in checks if c.is_violation]

```

if violations:

```
violations.sort(  
    key=lambda v: {"critical": 0, "high": 1, "medium": 2, "low": 3}  
    .get(v.severity, 4)  
)  
return violations[0]
```

return ViolationResult(is_violation=False)

def _check_data_access(self, event: TrajectoryEvent, spec: dict) -> ViolationResult:

"""Check if data access violates allowed patterns."""

if "k8s_get" not in event.action and "k8s_list" not in event.action:

return ViolationResult(is_violation=False)

allowed = spec.get("allowed_behaviors", {}).get("data_access", {})

denied_paths = allowed.get("denied_paths", [])

resource = event.resource.lower()

for pattern in denied_paths:

if self._match_glob(resource, pattern):

```
return ViolationResult(  
    is_violation=True,  
    violation_type="weight_access_outside_pipeline",  
    severity="critical",  
    description=f"Access to denied path: {event.resource}",  
    spec_name=spec["name"],  
    event=event,  
    step_number=event.trajectory_step,  
)
```

return ViolationResult(is_violation=False)

def _check_network_egress(self, event: TrajectoryEvent, spec: dict) -> ViolationResult:

"""Check for suspicious outbound network activity."""

if "network" not in event.action:

return ViolationResult(is_violation=False)

denied = (

spec.get("allowed_behaviors", {})

.get("network", {})

.get("denied_egress", [])

)

destination = event.details.get("destination", "")

for pattern in denied:

if self._match_glob(destination, pattern):

```
return ViolationResult(  
    is_violation=True,  
    violation_type="weight_access_outside_pipeline",  
    severity="critical",  
    description=f"Access to denied path: {event.resource}",  
    spec_name=spec["name"],  
    event=event,  
    step_number=event.trajectory_step,  
)
```

```

        is_violation=True,
        violation_type="suspicious_egress",
        severity="critical",
        description=f'Egress to denied destination: {destination}',
        spec_name=spec["name"],
        event=event,
        step_number=event.trajectory_step,
    )

```

```

    return ViolationResult(is_violation=False)

```

```

def _check_gpu_anomaly(self, event: TrajectoryEvent, spec: dict) -> ViolationResult:

```

```

    """Check GPU metrics against expected profile."""

```

```

    if event.source.value != "dcgm_gpu":

```

```

        return ViolationResult(is_violation=False)

```

```

    gpu_spec = spec.get("allowed_behaviors", {}).get("gpu_profile", {})

```

```

    z_score = event.details.get("z_score", 0)

```

```

    if z_score > 3.0:

```

```

        return ViolationResult(

```

```

            is_violation=True,

```

```

            violation_type="unusual_gpu_pattern",

```

```

            severity="high" if z_score > 5 else "medium",

```

```

            description=(

```

```

                f'GPU metric {event.details.get('metric', 'unknown')} '

```

```

                f'deviated {z_score:.1f}σ from baseline"

```

```

            ),

```

```

            spec_name=spec["name"],

```

```

            event=event,

```

```

            step_number=event.trajectory_step,

```

```

        )

```

```

    return ViolationResult(is_violation=False)

```

```

def _check_checkpoint_behavior(self, event: TrajectoryEvent, spec: dict) -> ViolationResult:

```

```

    """Check checkpoint creation patterns."""

```

```

    if "checkpoint" not in event.action.lower():

```

```

        return ViolationResult(is_violation=False)

```

```

    ckpt_spec = spec.get("allowed_behaviors", {}).get("checkpoints", {})

```

```

    allowed_formats = ckpt_spec.get("allowed_formats", [])

```

```

    resource = event.resource.lower()

```

```

    if allowed_formats and not any(resource.endswith(f) for f in allowed_formats):

```

```

        return ViolationResult(

```

```

            is_violation=True,

```

```
violation_type="checkpoint_anomaly",
severity="medium",
description=f"Checkpoint in unexpected format: {event.resource}",
spec_name=spec["name"],
event=event,
step_number=event.trajectory_step,
)
```

```
return ViolationResult(is_violation=False)
```

```
@staticmethod
```

```
def _match_glob(text: str, pattern: str) -> bool:
```

```
    """Simple glob matching (supports * wildcard)."""
```

```
    regex = pattern.replace(".", r"\.").replace("*", ".*")
```

```
    return bool(re.match(regex, text, re.IGNORECASE))
```

5.5 Cascaded Detector (HybridGuard)

```
python
```

```

# src/mlshield/detectors/cascade.py
import time
from dataclasses import dataclass
from typing import Optional
from ..ingestion.event_bus import TrajectoryEvent
from ..specs.spec_validator import SpecValidator, ViolationResult
from .layer1_rules import RuleEngine
from .layer2_ml import MLDetector
from .layer3_llm import LLMJudge
from ..metrics.temporal import TemporalMetrics

@dataclass
class DetectionResult:
    """Final output of the cascaded detector."""
    event: TrajectoryEvent
    is_threat: bool
    confidence: float      # 0.0 - 1.0
    threat_type: str
    severity: str
    description: str
    detected_by_layer: int  # 1, 2, or 3
    detection_latency_ms: float
    step_detected: int      # Which trajectory step
    explanation: str = ""   # Natural language (from Layer 3)

class CascadedDetector:
    """
    3-layer cascaded detector adapted from StepShield's HybridGuard.

    Layer 1 (Static Rules): Microsecond-latency hard policy checks.
                           Clears ~95% of benign events.
    Layer 2 (ML Model):   Millisecond-latency anomaly detection on
                           escalated events using LSTM + Isolation Forest.
    Layer 3 (LLM Judge):  Second-latency semantic analysis for the most
                           complex cases (<1% of events).
    """

    def __init__(
        self,
        spec_validator: SpecValidator,
        ml_detector: MLDetector,
        llm_judge: LLMJudge,
        layer2_threshold: float = 0.6,
        layer3_threshold: float = 0.8,
    ):
        self.spec_validator = spec_validator

```



```
self.layer1 = RuleEngine(spec_validator)
self.layer2 = ml_detector
self.layer3 = llm_judge
self.layer2_threshold = layer2_threshold
self.layer3_threshold = layer3_threshold
self.temporal_metrics = TemporalMetrics()
```

```
# Counters for cascade efficiency
```

```
self.total_events = 0
self.layer1_cleared = 0
self.layer2_processed = 0
self.layer3_processed = 0
```

```
async def evaluate(self, event: TrajectoryEvent) -> DetectionResult:
```

```
    """Run event through the cascade."""
```

```
    self.total_events += 1
    start = time.perf_counter()
```

```
# ——— Layer 1: Static Rules ———
```

```
l1_result = self.layer1.check(event)
elapsed = (time.perf_counter() - start) * 1000
```

```
if l1_result.is_violation and l1_result.severity == "critical":
```

```
    # Hard violation — immediate alert, skip other layers
```

```
    result = DetectionResult(
        event=event,
        is_threat=True,
        confidence=0.95,
        threat_type=l1_result.violation_type,
        severity=l1_result.severity,
        description=l1_result.description,
        detected_by_layer=1,
        detection_latency_ms=elapsed,
        step_detected=event.trajectory_step,
    )
    self.temporal_metrics.record_detection(result)
    return result
```

```
if not l1_result.is_violation:
```

```
    # Benign — check if rules are confident enough to clear
```

```
    self.layer1_cleared += 1
    return DetectionResult(
        event=event,
        is_threat=False,
        confidence=0.0,
        threat_type="",
        severity="info",
```

```
description="Cleared by Layer 1",
detected_by_layer=1,
detection_latency_ms=elapsed,
step_detected=event.trajectory_step,
)
```

```
# —— Layer 2: ML Anomaly Detection ——
```

```
self.layer2_processed += 1
```

```
l2_score = await self.layer2.score(event)
```

```
elapsed = (time.perf_counter() - start) * 1000
```

```
if l2_score < self.layer2_threshold:
```

```
    return DetectionResult(
        event=event,
        is_threat=False,
        confidence=l2_score,
        threat_type="",
        severity="info",
        description=f"Cleared by Layer 2 (score={l2_score:.3f})",
        detected_by_layer=2,
        detection_latency_ms=elapsed,
        step_detected=event.trajectory_step,
    )
```

```
if l2_score >= self.layer3_threshold:
```

```
    # High enough confidence to alert without LLM
```

```
    result = DetectionResult(
        event=event,
        is_threat=True,
        confidence=l2_score,
        threat_type=l1_result.violation_type or "ml_anomaly",
        severity="high",
        description=f"ML detector flagged (score={l2_score:.3f})",
        detected_by_layer=2,
        detection_latency_ms=elapsed,
        step_detected=event.trajectory_step,
    )
```

```
    self.temporal_metrics.record_detection(result)
```

```
    return result
```

```
# —— Layer 3: LLM Semantic Judge ——
```

```
self.layer3_processed += 1
```

```
l3_result = await self.layer3.judge(event, l1_result, l2_score)
```

```
elapsed = (time.perf_counter() - start) * 1000
```

```
result = DetectionResult(
    event=event,
```

```

is_threat=l3_result["is_threat"],
confidence=l3_result["confidence"],
threat_type=l3_result.get("threat_type", "complex_anomaly"),
severity=l3_result.get("severity", "medium"),
description=l3_result.get("description", ""),
detected_by_layer=3,
detection_latency_ms=elapsed,
step_detected=event.trajectory_step,
explanation=l3_result.get("explanation", ""),
)

```

```

if result.is_threat:
    self.temporal_metrics.record_detection(result)
return result

```

```

def get_cascade_stats(self) -> dict:
    """Return cascade efficiency metrics."""
    return {
        "total_events": self.total_events,
        "layer1_cleared_pct": (
            self.layer1_cleared / max(self.total_events, 1) * 100
        ),
        "layer2_processed_pct": (
            self.layer2_processed / max(self.total_events, 1) * 100
        ),
        "layer3_processed_pct": (
            self.layer3_processed / max(self.total_events, 1) * 100
        ),
    }

```

5.6 Layer 2: LSTM Sequence Detector

python

```

# src/mlshield/detectors/models/lstm_detector.py
import torch
import torch.nn as nn
import numpy as np
from typing import Optional

class TrajectoryLSTM(nn.Module):
    """LSTM for detecting anomalous event sequences in ML infrastructure."""

    def __init__(
        self,
        input_dim: int = 32,    # Feature dimension per event
        hidden_dim: int = 64,
        num_layers: int = 2,
        dropout: float = 0.2,
    ):
        super().__init__()
        self.lstm = nn.LSTM(
            input_dim, hidden_dim, num_layers,
            batch_first=True, dropout=dropout
        )
        self.attention = nn.Linear(hidden_dim, 1)
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, 32),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(32, 1),
            nn.Sigmoid(),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: (batch, seq_len, input_dim) — sequence of event features
        Returns:
            anomaly_scores: (batch, 1) — anomaly probability
        """
        lstm_out, _ = self.lstm(x)          # (batch, seq_len, hidden)
        attn_weights = torch.softmax(
            self.attention(lstm_out), dim=1
        )                                   # (batch, seq_len, 1)
        context = (lstm_out * attn_weights).sum(dim=1) # (batch, hidden)
        return self.classifier(context)

```

```

class EventFeaturizer:

```

```
"""Convert TrajectoryEvents into numeric feature vectors."""
```

```
# Map action types to numeric categories
```

```
ACTION_CATEGORIES = {  
    "k8s_get": 0, "k8s_list": 1, "k8s_create": 2,  
    "k8s_update": 3, "k8s_delete": 4, "k8s_exec": 5,  
    "k8s_attach": 6, "k8s_portforward": 7,  
    "gpu_metrics_snapshot": 8, "gpu_anomaly": 9,  
    "network_egress": 10, "checkpoint_create": 11,  
    "weight_access": 12, "unknown": 13,  
}
```

```
def featurize(self, event) -> np.ndarray:
```

```
    """Convert a single event to a 32-dim feature vector."""
```

```
    features = np.zeros(32, dtype=np.float32)
```

```
# Action category (one-hot, dims 0-13)
```

```
action = event.action.lower()
```

```
for key, idx in self.ACTION_CATEGORIES.items():
```

```
    if key in action:
```

```
        features[idx] = 1.0
```

```
        break
```

```
# Time features (dims 14-16)
```

```
features[14] = event.timestamp.hour / 24.0
```

```
features[15] = event.timestamp.minute / 60.0
```

```
features[16] = 1.0 if event.timestamp.weekday() >= 5 else 0.0 # Weekend
```

```
# Security signals (dims 17-22)
```

```
details = event.details or {}
```

```
features[17] = 1.0 if details.get("is_weight_access") else 0.0
```

```
features[18] = min(details.get("z_score", 0) / 10.0, 1.0)
```

```
features[19] = 1.0 if "exec" in action or "attach" in action else 0.0
```

```
features[20] = 1.0 if details.get("response_code", 200) >= 400 else 0.0
```

```
features[21] = min(event.trajectory_step / 100.0, 1.0)
```

```
# GPU metrics (dims 22-27) — if available
```

```
features[22] = min(details.get("DCGM_FI_DEV_GPU_UTIL", 0) / 100, 1.0)
```

```
features[23] = min(details.get("DCGM_FI_DEV_MEM_COPY_UTIL", 0) / 100, 1.0)
```

```
features[24] = min(details.get("DCGM_FI_DEV_FB_USED", 0) / 80000, 1.0) # 80GB max
```

```
features[25] = min(details.get("DCGM_FI_DEV_GPU_TEMP", 0) / 90, 1.0)
```

```
features[26] = min(details.get("DCGM_FI_DEV_POWER_USAGE", 0) / 700, 1.0)
```

```
features[27] = min(details.get("DCGM_FI_DEV_ENC_UTIL", 0) / 100, 1.0)
```

```
# Source encoding (dims 28-31)
```

```
source_map = {"k8s_audit": 28, "dcmg_gpu": 29, "app_event": 30, "falco_alert": 31}
```

```
src_idx = source_map.get(event.source.value, 30)
```

```
features[src_idx] = 1.0
```

```
return features
```

5.7 Layer 3: LLM Semantic Judge

```
python
```

```
# src/mlshield/detectors/layer3_llm.py
```

```
import httpx
```

```
import json
```

```
class LLMJudge:
```

```
    """Uses an LLM to provide semantic analysis of complex security events."""
```

```
    def __init__(self, api_url: str = "https://api.anthropic.com/v1/messages"):
```

```
        self.api_url = api_url
```

```
    async def judge(self, event, l1_result, l2_score: float) -> dict:
```

```
        """
```

```
        Provide semantic judgment on a potentially malicious event.
```

```
        Only called for <1% of events (Layer 2 medium-confidence zone).
```

```
        """
```

```
        prompt = self._build_prompt(event, l1_result, l2_score)
```

```
        try:
```

```
            async with httpx.AsyncClient() as client:
```

```
                response = await client.post(
```

```
                    self.api_url,
```

```
                    headers={"Content-Type": "application/json"},
```

```
                    json={
```

```
                        "model": "claude-sonnet-4-20250514",
```

```
                        "max_tokens": 500,
```

```
                        "messages": [{"role": "user", "content": prompt}],
```

```
                    },
```

```
                    timeout=10.0,
```

```
                )
```

```
                data = response.json()
```

```
                text = data["content"][0]["text"]
```

```
            return self._parse_response(text)
```

```
        except Exception:
```

```
            # Fallback: use Layer 2 score directly
```

```
            return {
```

```
                "is_threat": l2_score > 0.7,
```

```
                "confidence": l2_score,
```

```
                "severity": "medium",
```

```
                "description": "LLM judge unavailable, using ML score",
```

```
                "explanation": "",
```

```
            }
```

```
    def _build_prompt(self, event, l1_result, l2_score: float) -> str:
```

```
        return f"""You are a security analyst for an ML training cluster.
```

```
Analyze this event and determine if it indicates a security threat.
```

EVENT:

- Action: {event.action}
- Resource: {event.resource}
- User: {event.user}
- Job ID: {event.job_id}
- Timestamp: {event.timestamp}
- Details: {json.dumps(event.details, default=str)}

CONTEXT:

- Layer 1 (rule engine) flagged: {l1_result.violation_type or 'no violation'}
- Layer 2 (ML detector) anomaly score: {l2_score:.3f}
- This is step {event.trajectory_step} in the job's trajectory

Respond with ONLY a JSON object:

```
{{  
  "is_threat": true/false,  
  "confidence": 0.0-1.0,  
  "threat_type": "weight_exfiltration|data_poisoning|unauthorized_access|benign",  
  "severity": "critical|high|medium|low",  
  "description": "one-line summary",  
  "explanation": "2-3 sentence reasoning"  
}}
```

```
def _parse_response(self, text: str) -> dict:  
    try:  
        clean = text.strip().strip("`json").strip("`").strip()  
        return json.loads(clean)  
    except json.JSONDecodeError:  
        return {  
            "is_threat": False,  
            "confidence": 0.5,  
            "severity": "medium",  
            "description": "Could not parse LLM response",  
            "explanation": text[:200],  
        }
```

5.8 Temporal Metrics (from StepShield)

python


```

# src/mlshield/metrics/temporal.py
from dataclasses import dataclass, field
from collections import defaultdict

@dataclass
class TemporalMetrics:
    """
    Temporal security metrics adapted from StepShield.
    Measures WHEN threats are detected, not just WHETHER.
    """
    detections: list = field(default_factory=list)
    _job_first_violation: dict = field(default_factory=dict)
    _job_first_detection: dict = field(default_factory=dict)

    def record_detection(self, result):
        """Record a detection event for temporal analysis."""
        self.detections.append(result)
        job_id = result.event.job_id
        step = result.step_detected

        # Track first actual violation vs first detection per job
        if job_id not in self._job_first_detection:
            self._job_first_detection[job_id] = step

    def record_ground_truth_violation(self, job_id: str, step: int):
        """Record when a violation actually began (for benchmark evaluation)."""
        if job_id not in self._job_first_violation:
            self._job_first_violation[job_id] = step

    def early_intervention_rate(self, max_acceptable_gap: int = 5) -> float:
        """
        ML-EIR: What fraction of threats were detected within
        `max_acceptable_gap` steps of the actual violation?
        Adapted from StepShield's EIR metric.
        """
        if not self._job_first_violation:
            return 0.0

        early_count = 0
        total = 0
        for job_id, violation_step in self._job_first_violation.items():
            if job_id in self._job_first_detection:
                total += 1
                detection_step = self._job_first_detection[job_id]
                gap = detection_step - violation_step
                if gap <= max_acceptable_gap:

```

```
early_count += 1
```

```
return early_count / max(total, 1)
```

```
def detection_gap(self) -> dict:
```

```
    """Average and median steps between violation and detection."""
```

```
    import numpy as np
```

```
    gaps = []
```

```
    for job_id, violation_step in self._job_first_violation.items():
```

```
        if job_id in self._job_first_detection:
```

```
            gaps.append(self._job_first_detection[job_id] - violation_step)
```

```
    if not gaps:
```

```
        return {"mean": 0, "median": 0, "max": 0}
```

```
    return {
```

```
        "mean": float(np.mean(gaps)),
```

```
        "median": float(np.median(gaps)),
```

```
        "max": int(np.max(gaps)),
```

```
    }
```

```
def damage_prevented(self) -> float:
```

```
    """
```

```
    Estimate % of potential damage prevented by early detection.
```

```
    Assumes linear damage accumulation — earlier detection = more prevented.
```

```
    """
```

```
    if not self._job_first_violation:
```

```
        return 0.0
```

```
    prevented_scores = []
```

```
    max_trajectory_length = 100 # Assume 100-step trajectories
```

```
    for job_id, violation_step in self._job_first_violation.items():
```

```
        if job_id in self._job_first_detection:
```

```
            detection_step = self._job_first_detection[job_id]
```

```
            remaining = max_trajectory_length - detection_step
```

```
            total_damage_window = max_trajectory_length - violation_step
```

```
            if total_damage_window > 0:
```

```
                prevented_scores.append(remaining / total_damage_window)
```

```
    return sum(prevented_scores) / max(len(prevented_scores), 1)
```

```
def summary(self) -> dict:
```

```
    """Full temporal metrics summary."""
```

```
    return {
```

```
        "total_detections": len(self.detections),
```

```
        "early_intervention_rate": self.early_intervention_rate(),
```

```
        "detection_gap": self.detection_gap(),
```

```
    "damage_prevented_pct": self.damage_prevented() * 100,  
    "detections_by_layer": self._count_by_layer(),  
}
```

```
def _count_by_layer(self) -> dict:  
    counts = defaultdict(int)  
    for d in self.detections:  
        counts[f"layer_{d.detected_by_layer}"] += 1  
    return dict(counts)
```

5.9 Synthetic Benchmark Generator

python

```

# benchmark/generate_dataset.py
"""
Generate synthetic ML infrastructure security trajectories.
Produces labeled data for training and evaluating MLShield.
"""

import json
import random
from datetime import datetime, timedelta
from pathlib import Path

def generate_normal_training_trajectory(job_id: str, num_steps: int = 50) -> list:
    """Generate a normal training job trajectory."""
    events = []
    base_time = datetime.utcnow()

    for step in range(num_steps):
        t = base_time + timedelta(seconds=step * 30)
        event_type = random.choices(
            ["gpu_snapshot", "checkpoint", "data_read", "log_write", "health_check"],
            weights=[40, 5, 20, 25, 10],
        )[0]

        event = {
            "job_id": job_id,
            "step": step,
            "timestamp": t.isoformat(),
            "is_malicious": False,
            "violation_type": None,
        }

        if event_type == "gpu_snapshot":
            event.update({
                "action": "gpu_metrics_snapshot",
                "resource": f"gpu/0",
                "details": {
                    "DCGM_FI_DEV_GPU_UTIL": random.gauss(85, 5),
                    "DCGM_FI_DEV_MEM_COPY_UTIL": random.gauss(70, 8),
                    "DCGM_FI_DEV_FB_USED": random.gauss(60000, 5000),
                    "DCGM_FI_DEV_GPU_TEMP": random.gauss(65, 3),
                }
            })

        elif event_type == "checkpoint":
            event.update({
                "action": "k8s_create",
                "resource": f"persistentvolumeclaims/checkpoint-{step}.safetensors",
                "details": {"is_weight_access": True, "size_gb": random.uniform(5, 30)}
            })

```

```

    })
    elif event_type == "data_read":
        event.update({
            "action": "k8s_get",
            "resource": "pods/data-loader",
            "details": {"path": f"/data/training/batch_{step}.parquet"}
        })
    else:
        event.update({
            "action": "k8s_get",
            "resource": "pods/health",
            "details": {}
        })

    events.append(event)
return events

def generate_weight_exfiltration_trajectory(
    job_id: str, attack_start_step: int = 20, num_steps: int = 50
) -> list:
    """Generate a trajectory with weight exfiltration attack."""
    events = generate_normal_training_trajectory(job_id, num_steps)

    # Inject malicious events starting at attack_start_step
    for i in range(attack_start_step, min(attack_start_step + 15, num_steps)):
        phase = i - attack_start_step

        if phase < 3:
            # Reconnaissance: list model files
            events[i].update({
                "action": "k8s_list",
                "resource": "persistentvolumeclaims/models-production",
                "details": {"is_weight_access": True, "path": "/models/production/"},
                "is_malicious": True,
                "violation_type": "weight_access_outside_pipeline",
            })
        elif phase < 7:
            # Staging: access production weights
            events[i].update({
                "action": "k8s_get",
                "resource": f"persistentvolumeclaims/frontier-model-v3.safetensors",
                "details": {
                    "is_weight_access": True,
                    "path": "/models/production/frontier-v3.safetensors",
                    "size_gb": 45,
                },
            })

```

```

        "is_malicious": True,
        "violation_type": "weight_access_outside_pipeline",
    })
elif phase < 10:
    # Conversion: convert to portable format
    events[i].update({
        "action": "k8s_exec",
        "resource": "pods/training-job",
        "details": {
            "command": "python convert_to_onnx.py",
            "is_weight_access": True,
        },
        "is_malicious": True,
        "violation_type": "unauthorized_conversion",
    })
else:
    # Exfiltration: upload to external storage
    events[i].update({
        "action": "network_egress",
        "resource": "pods/training-job",
        "details": {
            "destination": "attacker-bucket.s3.amazonaws.com",
            "bytes_sent": random.randint(1_000_000_000, 5_000_000_000),
        },
        "is_malicious": True,
        "violation_type": "suspicious_egress",
    })

```

```

return events

```

```

def generate_shadowray_attack_trajectory(job_id: str, num_steps: int = 50) -> list:

```

```

    """Generate a ShadowRay-style cluster compromise trajectory."""

```

```

    events = generate_normal_training_trajectory(job_id, num_steps)

```

```

    attack_start = 15

```

```

    for i in range(attack_start, min(attack_start + 20, num_steps)):

```

```

        phase = i - attack_start

```

```

        if phase < 5:

```

```

            # Ray dashboard exploit

```

```

            events[i].update({
                "action": "k8s_exec",
                "resource": "pods/ray-head",
                "details": {"source_ip": "10.0.0.99", "user_agent": "python-requests"},
                "is_malicious": True,
                "violation_type": "unauthorized_access",
            })

```

```

    })
elif phase < 10:
    # Lateral movement
    events[i].update({
        "action": "k8s_get",
        "resource": "secrets/aws-credentials",
        "details": {"namespace": "ml-training"},
        "is_malicious": True,
        "violation_type": "credential_theft",
    })
else:
    # Cryptojacking / resource abuse
    events[i].update({
        "action": "gpu_metrics_snapshot",
        "resource": "gpu/0",
        "details": {
            "DCGM_FI_DEV_GPU_UTIL": random.gauss(99, 0.5), # Maxed out
            "DCGM_FI_DEV_GPU_TEMP": random.gauss(88, 2), # Overheating
            "z_score": 6.5,
        },
        "is_malicious": True,
        "violation_type": "resource_abuse",
    })

return events


def generate_full_dataset(
    n_normal: int = 1500,
    n_exfil: int = 200,
    n_shadowray: int = 150,
    n_distill: int = 150,
    output_dir: str = "benchmark/data",
):
    """Generate the full benchmark dataset."""
    Path(output_dir).mkdir(parents=True, exist_ok=True)
    dataset = []

    for i in range(n_normal):
        trajectory = generate_normal_training_trajectory(f"normal-{i}")
        dataset.append({"job_id": f"normal-{i}", "label": "benign", "events": trajectory})

    for i in range(n_exfil):
        attack_step = random.randint(10, 30)
        trajectory = generate_weight_exfiltration_trajectory(
            f"exfil-{i}", attack_start_step=attack_step
        )

```

```

dataset.append({
    "job_id": f'exfil-{i}',
    "label": "weight_exfiltration",
    "attack_start_step": attack_step,
    "events": trajectory,
})

for i in range(n_shadowray):
    trajectory = generate_shadowray_attack_trajectory(f'shadowray-{i}')
    dataset.append({
        "job_id": f'shadowray-{i}',
        "label": "cluster_compromise",
        "attack_start_step": 15,
        "events": trajectory,
    })

random.shuffle(dataset)

with open(f'{output_dir}/mlshield_benchmark_v1.json', "w") as f:
    json.dump(dataset, f, indent=2, default=str)

print(f'Generated {len(dataset)} trajectories:')
print(f' Normal: {n_normal}')
print(f' Weight exfiltration: {n_exfil}')
print(f' ShadowRay-style: {n_shadowray}')
print(f' Unauthorized distillation: {n_distill}')
print(f' Saved to: {output_dir}/mlshield_benchmark_v1.json')

if __name__ == "__main__":
    generate_full_dataset()

```

5.10 FastAPI Dashboard

```
python
```



```
# src/mlshield/api/app.py
from fastapi import FastAPI
from fastapi.responses import HTMLResponse
from prometheus_client import make_asgi_app, Counter, Histogram, Gauge

app = FastAPI(title="MLShield", version="0.1.0")

# Prometheus metrics
EVENTS_PROCESSED = Counter("mlshield_events_total", "Total events processed", ["layer"])
THREATS_DETECTED = Counter("mlshield_threats_total", "Threats detected", ["type", "severity"])
DETECTION_LATENCY = Histogram("mlshield_detection_latency_ms", "Detection latency in ms")
EARLY_INTERVENTION_RATE = Gauge("mlshield_eir", "Early Intervention Rate")
CASCADE_EFFICIENCY = Gauge("mlshield_cascade_layer1_pct", "% cleared by Layer 1")

# Mount Prometheus metrics endpoint
metrics_app = make_asgi_app()
app.mount("/metrics", metrics_app)

@app.get("/", response_class=HTMLResponse)
async def dashboard():
    return """
    <html>
    <head><title>MLShield Dashboard</title></head>
    <body>
        <h1>  MLShield — ML Infrastructure Security Monitor</h1>
        <p>Endpoints:</p>
        <ul>
            <li><a href="/api/alerts">/api/alerts</a> — Recent alerts</li>
            <li><a href="/api/metrics/temporal">/api/metrics/temporal</a> — Temporal metrics</li>
            <li><a href="/api/metrics/cascade">/api/metrics/cascade</a> — Cascade efficiency</li>
            <li><a href="/api/health">/api/health</a> — Health check</li>
            <li><a href="/metrics">/metrics</a> — Prometheus endpoint</li>
            <li><a href="/docs">/docs</a> — API documentation</li>
        </ul>
    </body>
    </html>
    """

@app.get("/api/health")
async def health():
    return {"status": "healthy", "version": "0.1.0"}

@app.get("/api/alerts")
async def get_alerts(limit: int = 20):
    # In production: fetch from detector's detection history
    return {"alerts": [], "total": 0}
```

```
@app.get("/api/metrics/temporal")
async def get_temporal_metrics():
    # In production: fetch from TemporalMetrics instance
    return {
        "early_intervention_rate": 0.0,
        "detection_gap": {"mean": 0, "median": 0},
        "damage_prevented_pct": 0.0,
    }

@app.get("/api/metrics/cascade")
async def get_cascade_metrics():
    return {
        "layer1_cleared_pct": 95.0,
        "layer2_processed_pct": 4.0,
        "layer3_processed_pct": 1.0,
    }
```

6. BUILD SEQUENCE (What to do in what order)

Weekend 1: Foundation (Days 1-2)

1. Create repo structure and pyproject.toml
2. Implement `TrajectoryEvent` and `Trajectory` data models
3. Build the benchmark dataset generator (generate 2000 trajectories)
4. Write a basic `SpecValidator` with 3-4 hardcoded rules
5. **Milestone:** `python -m benchmark.generate_dataset` produces labeled data

Week 1: Layer 1 + Ingestion (Days 3-7)

6. Build K8s audit log ingester (file-based for now)
7. Build DCGM metrics ingester (mock data for now)
8. Implement Layer 1 static rule engine
9. Write tests for rule engine against benchmark data
10. **Milestone:** Layer 1 processes benchmark trajectories, flags known violations

Week 2: Layer 2 + Temporal Metrics (Days 8-14)

11. Implement `EventFeaturizer`
12. Train LSTM on benchmark normal vs. malicious trajectories
13. Implement Isolation Forest for GPU telemetry anomalies

14. Implement `TemporalMetrics` (EIR, Detection Gap, Damage Prevented)
15. **Milestone:** Layer 2 trained and evaluated on benchmark

Week 3: Layer 3 + API + Polish (Days 15-21)

16. Implement LLM semantic judge (with API key)
17. Build `CascadedDetector` orchestrating all 3 layers
18. Build FastAPI dashboard and Prometheus metrics
19. Create `docker-compose.yml` for one-command demo
20. Write README with architecture diagram
21. **Milestone:** `docker-compose up` runs full demo

Week 4 (stretch): Evaluation + Paper

22. Run full benchmark evaluation
 23. Compare against Falco baseline (same data, Falco rules only)
 24. Generate evaluation charts (EIR comparison, cascade efficiency)
 25. Write up results
-

7. HARDWARE / COST REQUIREMENTS

Minimum (free tier):

- **Laptop/desktop** with Python 3.10+ — sufficient for all development
- **No GPU needed** for the MVP — LSTM training on 2000 trajectories runs on CPU in minutes
- **Docker Desktop** for the demo stack
- **Minikube or kind** for local K8s (free, runs on laptop)
- **DCGM:** Mock the metrics for demo. Real DCGM needs an NVIDIA GPU, but you can simulate the Prometheus output format

For a more realistic demo:

- **AWS free tier** or a small EC2 instance with GPU (~\$0.50-3/hr spot)
- Or **Google Colab** (free GPU) for training the LSTM

API costs:

- **Anthropic API** for Layer 3: ~\$0.01-0.10 per complex event judged (only <1% of events reach Layer 3, so total cost is minimal during development)
-

8. WHAT TO PUT IN THE README

```
markdown

#  MLShield

**ML-Infrastructure-Aware Anomaly Detection for Model Weight Exfiltration**

MLShield monitors Kubernetes-based ML training clusters for model weight
exfiltration using a cascaded detection architecture adapted from
[StepShield](https://arxiv.org/abs/2601.22136).

It complements inference-output verification approaches (like
[Warr et al. 2025](https://arxiv.org/abs/2511.02620)) by covering
non-inference exfiltration channels: insider threats, side channels,
and compromised infrastructure.

## Why MLShield?

Existing container security tools (Falco, Tetragon) have **zero ML awareness**.
ML monitoring tools (DCGM, Prometheus) track performance but **not security**.
MLShield bridges this gap.

## Quick Start

git clone https://github.com/HemantBK/mlshield.git
cd mlshield
docker-compose up

Then open http://localhost:8000 for the dashboard.

## Architecture

[diagram here]

## Citing

If you use MLShield in your research, please cite:
[bibtex here]
```

9. KEY RISKS AND MITIGATIONS

Risk	Mitigation
No access to real ML cluster	Use minikube + synthetic benchmark. Keri will understand.

Risk	Mitigation
LSTM overfits on synthetic data	Use simple features, regularization, and validate on held-out scenarios
Layer 3 LLM costs money	Use it sparingly (<1% of events). Mock it for most testing.
Scope creep	Ship Layer 1 + benchmark first. Everything else is additive.
DCGM needs real GPU	Mock the Prometheus endpoint format. Document real GPU setup.

10. SUCCESS CRITERIA

Your MVP is "good enough" when:

- ☐ `python benchmark/generate_dataset.py` produces 2000+ labeled trajectories
- ☐ Layer 1 catches all critical violations in the benchmark (100% recall on hard rules)
- ☐ Layer 2 LSTM achieves >85% AUC on held-out test trajectories
- ☐ Temporal metrics show measurably better EIR than a Falco-only baseline
- ☐ `docker-compose up` launches the full stack in one command
- ☐ README explains the project clearly with architecture diagram
- ☐ Code is clean, typed, and has basic tests